

MEDIATEK

MT8676 Yocto System User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	彭凯	正式版

目录

版本记录	2
目录	3
图片目录	4
表格目录	4
1 System	6
1.1 名词解释	6
1.2 Yocto 系统架构	7
1.2.1 软件架构	7
1.2.2 Yocto Meta Layers	7
1.2.3 代码结构	9
1.2.4 常用文件路径	10
1.2.5 分区表	12
1.3 Yocto 编译与烧录	13
1.3.1 Build Server	14
1.3.2 编译软件	14
1.3.3 软件构成	15
1.3.4 烧录软件	16
1.3.5 编译 SDK	18
1.3.6 使用 SDK	18
1.4 Yocto 客制化	19
1.4.1 常用变量	19
1.4.2 bb 文件的基本结构	20
1.4.3 开机自启动	21
1.4.4 依赖	22
1.4.5 树外驱动	23
1.4.6 image bb	26
1.4.7 fog 简介	26
1.4.8 创建 project	27
1.4.9 创建 meta layer	27
1.5 Yocto Build Mode	28
1.5.1 使用方法	29

1.5.2 适配 29

1.6 Yocto 调试技巧 30

1.6.1 build 目录结构 30

1.6.2 工作目录常用子目录 31

1.6.3 打印变量值 32

1.6.4 加速 Yocto 编译 33

1.7 开机流程 34

1.7.1 BootROM 34

1.7.2 LK2 34

1.7.3 Kernel 35

1.7.4 initramfs 36

1.7.5 systemd 36

1.8 Yocto 常用调试工具 37

附件一 附加条款 39

图片目录

图 1-1. Yocto 软件架构 7

图 1-2. MT8676 meta layers 8

图 1-3. 代码结构 9

图 1-4. 下载 SP_Flash_Tool 16

图 1-5. 加载 flash.xml 17

图 1-6. 软件烧录 17

图 1-7. 安装 SDK 18

图 1-8. 目录结构 19

图 1-9. meta layer 目录结构 28

图 1-10. 编译完成后的目录结构 31

图 1-11. 开机流程 34

图 1-12. kernel_init 36

表格目录

表 1-1. 名词解释 6

表 1-2. Meta layer 说明 8

表 1-3. 主要模块路径 10

表 1-4. 常用文件路径.....10

表 1-5. 分区表格式.....11

表 1-6. ko table 格式.....12

表 1-7. 分区表12

表 1-8. Build server 要求14

表 1-9. 镜像文件说明.....15

表 1-10. Yocto 常用变量.....19

表 1-11. 创建 project27

表 1-12. Yocto build mode28

表 1-13. Build variant29

表 1-14. build 目录常用文件和目录.....31

表 1-15. 工作目录常用子目录.....32

表 1-16. 常见服务37

表 1-17. Yocto 常用调试工具.....37

1 System

本章节主要介绍 MT8676 Yocto 系统相关内容以及常见系统问题的处理方法。

1.1 名词解释

表 1-1. 名词解释

缩略词	全称及释义
bb	Bitbake 配方文件（recipe），bb 文件描述了如何构建特定软件包。
Bitbake	Yocto 项目的核心构建工具，解析 bb 和 bbclass 文件，并执行任务。
CCU	Camera Control Unit，相机控制单元。
DPM	DRAM Power Manager，处理 SPM 的指令，控制 DRAM 的速度、进出 low power 模式等。
Layer	即元数据层（meta layer），Yocto 使用 layer 来管理 bb 和配置文件。Layer 之间可独立开发和维护。
MCUPM	CPU 专属的 power/performance manager。
NeuroPilot	NeuroPilot ¹ 是一套用于在 MediaTek 平台开发高效 AI 应用程序的软件工具和 API。
OE	OpenEmbedded，提供了构建嵌入式 Linux 系统的基本组件，包含了常用的工具链、库和应用程序。
Poky	Yocto 项目的参考发行版。它是一个完整的构建系统，可以用来创建嵌入式 Linux 镜像。
SCP	System Companion Processor，在系统处于低功耗状态时执行始终开启任务的子系统。
SPM	System Power Management，管理系统的电源。
SSPM	Secure System Power Manager，控制 SoC 电源并驻留在安全域中的子系统。
T-box	Telematics-BOX，车联网控制单元。
tinysys	指除主处理器之外的其他处理器，例如 APUSYS。
VCP	Video Controller Processor
Yocto	一个开源项目，可以为嵌入式系统创建自定义 Linux 发行版本。

¹ <https://neuropilot.mediatek.inc/>

1.2 Yocto 系统架构

1.2.1 软件架构

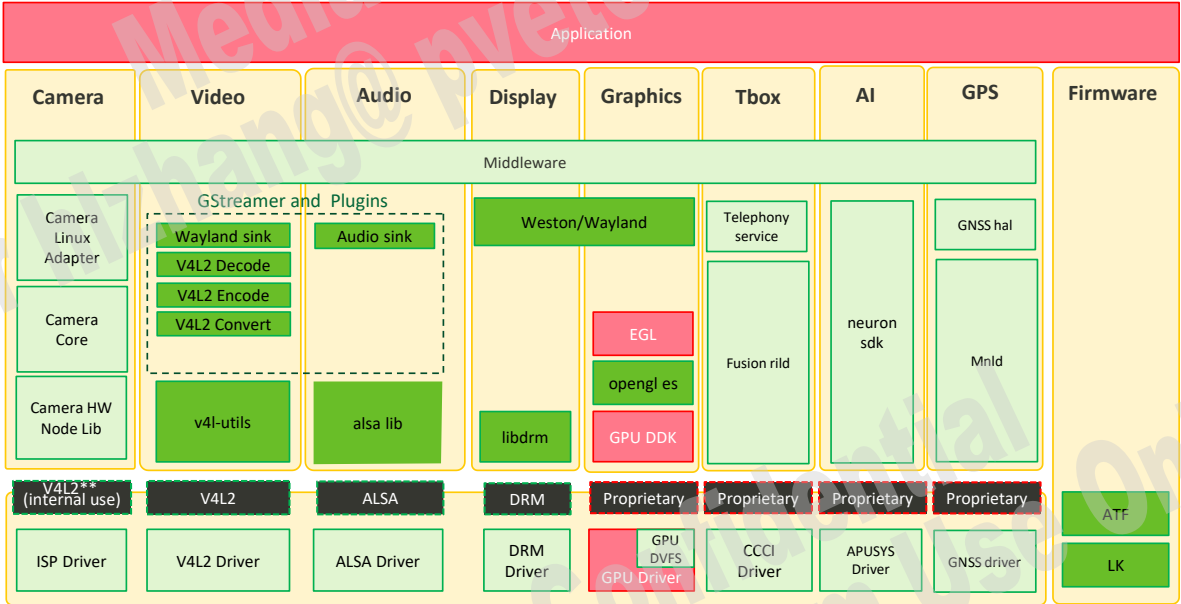


图 1-1. Yocto 软件架构

1.2.2 Yocto Meta Layers

MT8676 Yocto meta layers 如图 1-2 所示，主要由 Yocto core layer, OSS layer, MediaTek feature layer, MediaTek platform layer 4 部分构成。表 1-2 详细说明了每个 layer 包含的内容。

```
BBLAYERS ?= " \
    ##OEROOT##/./meta-mediatek-classes-overlay \
    ##OEROOT##/meta \
    ##OEROOT##/meta-poky \
    ##OEROOT##/meta-yocto-bsp \
    ##OEROOT##/./meta-openembedded/meta-fileystems \
    ##OEROOT##/./meta-openembedded/meta-python \
    ##OEROOT##/./meta-openembedded/meta-multimedia \
    ##OEROOT##/./meta-openembedded/meta-networking \
    ##OEROOT##/./meta-openembedded/meta-oe \
    ##OEROOT##/./meta-clang \
    ##OEROOT##/./meta-mediatek \
    ##OEROOT##/./meta-mediatek-gpl \
    ##OEROOT##/./meta-mediatek-gplv2 \
    ##OEROOT##/./meta-mediatek-gstreamer \
    ##OEROOT##/./meta-mediatek-ml-np \
    ##OEROOT##/./meta-mediatek-mt8676 \
    ##OEROOT##/./meta-qt5 \
"
```

图 1-2. MT8676 meta layers

表 1-2. Meta layer 说明

Layer	Explanation
meta-mediatek-classes-overlay	覆盖 Yocto bbclass, 自定义 bbclass。
meta ²	包含 OpenEmbedded 的核心元数据, 不包含发行版本, 仅提供对模拟器的支持。
meta-poky	包含 Poky 发行版本的配置和元数据。
meta-yocto-bsp ³	包含 Yocto 项目参考硬件的 BSP。
meta-fileystems	包含文件系统相关的元数据, 例如 fuse、owfs、ntfs-3g 等。
meta-python	包含 Python 相关的元数据。
meta-multimedia	包含多媒体相关的元数据。
meta-networking	包含网络相关的元数据。
meta-oe	其他共享的 OE 元数据。
meta-clang	包含 clang/llvm 相关的元数据。
meta-qt5	包含 qt5 相关的元数据。
meta-mediatek	MediaTek 基础 layer, 包含 MediaTek 写的 bbclass 和与平台无关的软件包。
meta-mediatek-gpl	MediaTek gpl layer, 包含 u-boot 等软件包。

² <https://git.openembedded.org/openembedded-core/tree/README.OE-Core.md>

³ <https://git.yoctoproject.org/poky/tree/meta-yocto-bsp/README.hardware.md>

Layer	Explanation
meta-mediatek-gplv2	MediaTek gplv2 layer，包含 grep 等软件包的 GPLv2 版本。
meta-mediatek-gstreamer	MediaTek GStreamer layer。
meta-mediatek-ml-np	包含 MediaTek NeuroPilot 相关的软件包。
meta-mediatek-mt8676	MT8676 BSP layer，包含 MT8676 BSP 的配置文件和元数据。

1.2.3 代码结构

MT8676 Yocto 代码结构如图 1-3 和表 1-3 所示。

- meta 目录包含所有 Yocto 元数据，详细说明请参考 1.2.2 章节。
- patch 目录包含 MediaTek 对 OSS 打的补丁。
- prebuilt 目录包含预编译好的文件，例如可执行文件、动态库、镜像和工具链等。
- src 目录包含所有的源代码文件。

表 1-3 列出了主要模块的路径，及其简要说明。

meta	meta-clang
	meta-mediatek
	meta-mediatek-clang
	meta-mediatek-classes-overlay
	meta-mediatek-gpl
	meta-mediatek-gplv2
	meta-mediatek-gstreamer
	meta-mediatek-ml-np
	meta-mediatek-mt8676
	meta-openembedded
	meta-qt5
	meta-qt6
	meta-rust
	poky
patch	
prebuilt	
src	apps
	bach
	bsp
	connectivity
	devtools
	kernel
	ml
	modem
	multimedia
	navigation
	support
	telephony
	telephonyware
	tinysys

图 1-3. 代码结构

表 1-3. 主要模块路径

Layer	Explanation
meta/meta-mediatek-mt8676	MT8676 platform 层
src/bsp/lk2 src/bsp/dramk_8676	bootloader 源代码
src/kernel/linux/v6.1_mt8676/co_common	Linux Kernel 6.1 源代码
src/kernel/linux/v6.1_mt8676/co_device_module	MediaTek Linux kernel 6.1 设备驱动
src/kernel/modules	out-of-tree 设备驱动
src/apps/spm-base src/apps/atom-base	应用程序源代码
src/connectivity	Wi-Fi 相关的源代码
src/ml/neuropilot	NeuroPilot ⁴ 相关的源代码
src/telephony src/telephonyware	车机 tbox(telephony) data/call/sms FW 相关的源代码
src/tinysys	tinysys 相关的源代码

1.2.4 常用文件路径

表 1-4 列出了一些 Yocto 系统常用文件路径及其作用。

表 1-4. 常用文件路径

路径	说明
meta/meta-mediatek-mt8676/conf/machine/auto8676p1_64.conf	项目配置文件。
meta/meta-mediatek-mt8676/recipes-bsp/ptgen/files/auto8676p1_64/partition_table_emmc_ab.csv	分区表。分区表的格式参考 1.2.4.1 章节。
meta/meta-mediatek-mt8676/recipes-kernel/linux/ko_order_table/auto8676p1_64/ko_order_table.csv	ko table，决定 ko 加载顺序。格式参考 1.2.4.2 章节。
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/auto8676p1_64.dts	kernel dts 文件。
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/configs/auto8676p1_64_defconfig	kernel defconfig 文件。
src/devtools/dct/dws/mt6897/auto8676p1_64.dws	dws 文件，主要用于配置 GPIO。

⁴ <https://neuropilot.mediatek.inc/>

路径	说明
meta-mediatek-mt8676/recipes-core/base-files/base-files/auto8676p1_64/fstab	fstab 文件，控制分区挂载。
meta-mediatek-mt8676/recipes-auto/images/mtk-core-image-auto8676.bb	MT8676 镜像 bb，用于生成根文件系统。

1.2.4.1 分区表格式

分区表是一个 CSV 文件，可以用文本编辑器或 Excel 编辑。每一列的说明请参考表 1-5。

表 1-5. 分区表格式

表头	说明
Partition_Name	分区名字
Type	分区类型，EXT4 或 Raw data
Size_KB	分区大小，单位是 KB
Region(emmc)	分区所在的 eMMC 物理分区，EMMC_BOOT1、EMMC_BOOT2 或 EMMC_USER
Region(ufs)	分区所在的 UFS 物理分区，UFS_LU0、UFS_LU1（BOOT）或 UFS_LU2（USER）
Reserved	保留分区，Y 或 N
Download	是否烧录，Y 或 N。如果填 Y，Download_File 不能为空。
Download_File	烧录的文件名
OTA_Update	OTA 是否升级
EmptyBoot_Needed	未使用
Operation_Type	BOOTLOADERS：preloader 分区 AUTO：普通分区 PROTECTED：固件更新的备份分区 NEEDRESIZE：用户数据分区 RESERVED：与 Reserved 字段含义相同，保留分区
Default_Exist	Y 或 N，Y 表示分区存在

1.2.4.2 ko table 格式

ko table 是一个 CSV 文件，可以用文本编辑器或 Excel 编辑。每一列的说明请参考表 1-6。

表 1-6. ko table 格式

表头	说明
KO name (*.ko)	文件名，含扩展名.ko
KO path	此字段暂时未使用。
Vendor/Ramdisk in Normal(vendor/ramdisk)	可以填 vendor 或 ramdisk，表示 ko 放在 rootfs 或 ramdisk。
Need loaded before BootTime.completed(Y/N)	此字段暂时未使用。
Recovery(Y/N)	此字段暂时未使用。
Build mode	编译模式，表示指定模式是否使用。比如 user/userdebug/eng 表示三个模块都会使用；userdebug/eng 表示只有 userdebug 和 eng 模式会使用，user 模式不会使用。

1.2.5 分区表

表 1-7 列出了 MT8676 的分区表以及分区说明。

表 1-7. 分区表

Partition_Name	Type	Download_File	说明
preloader_a	Raw data	bl2.img	bootloader 分区。
pgpt	Raw data	NONE	GPT 分区表。
misc	Raw data	NONE	存储 OTA 升级过程中的一个标志位。
para	Raw data	NONE	存储 LK 和 kernel 使用的系统环境信息。
yocto-expdb	Raw data	NONE	用于存储 AEE DB。
nvcfg	EXT4	NONE	由各 module 直接透过 AP side FS 去存放数据。
nvdta	EXT4	NONE	存储 NVRAM 数据。
protect1	EXT4	NONE	存储 SIM lock 数据。
protect2	EXT4	NONE	存储 SIM lock 数据备份。
seccfg	Raw data	NONE	存储安全启动相关的信息。
otp	Raw data	NONE	eMMC OTP (one-time-program, e.g. IMEI)
pstore	Raw data	NONE	保存一些额外的日志，比如 power off charge。
modem_a	Raw data	modem.img	modem 镜像。

Partition_Name	Type	Download_File	说明
spmfw_a	Raw data	spmfw.img	spmfw 镜像。
mcf_ota_a	EXT4	mcf_ota.img	mfc_ota 镜像。
audio_dsp_a	Raw data	audio_dsp.img	audio_dsp 镜像。
pi_img_a	Raw data	pi_img.img	pi_img 镜像。
dpm_a	Raw data	dpm.img	dmp 镜像。
scp_a	Raw data	scp-fit.img	SCP 镜像。
ccu_a	Raw data	ccu.img	CCU 镜像。
vcp_a	Raw data	vcp.img	VCP 镜像。
sspm_a	Raw data	sspm.img	SSPM 镜像。
mcupm_a	Raw data	mcupm.img	MCUPM 镜像。
gpueb_a	Raw data	gpueb.img	GPUEB 镜像。
apusys_a	Raw data	apusys.img	APUSYS 镜像。
boot_a	Raw data	boot.img	Kernel 镜像。
tee_a	Raw data	tee.img	ATF 镜像。
connsys_bt_a	Raw data	connsys_bt.img	未使用，保留。
connsys_wifi_a	Raw data	connsys_wifi.img	未使用，保留。
connsys_gnss_a	Raw data	connsys_gnss.img	GNSS 镜像。
logo_a	Raw data	logo.img	Logo 镜像。
nvram	Raw data	NONE	NVDATA 备份分区。
boot_para	Raw data	NONE	存储 DRAM 校验数据。
dram_para	Raw data	NONE	存储 DRAM 校验数据。
system	EXT4	system.ext4	rootfs 镜像。
userdata	EXT4	userdata.ext4	用户数据。
sgpt	Raw data	NONE	GPT 分区备份。

1.3 Yocto 编译与烧录

本小节介绍了编译 Yocto 对主机的要求，软件的编译指令与烧录方法，以及 SDK 的编译指令和使用方法。

1.3.1 Build Server

表 1-8 列出了编译 MT8676 Yocto5.0 对编译主机的要求。

表 1-8. Build server 要求

项目	要求
磁盘空间	不小于 300GB
内存	不小于 32GB
发行版本	Ubuntu 20.04, 22.04 等，具体参考 Supported Linux Distributions ⁵ 。
编译主机需要安装的工具	参考 Required Packages for the Build Host ⁶ 。
Git	1.8.3.1 或更高版本
tar	1.28 或更高版本
Python	3.8.0 或更高版本
GNU Make	4.0 或更高版本

1.3.2 编译软件

编译 MT8676 Yocto 软件的指令如下所示。首先切换到 codebase 所在的目录，环境变量 TEMPLATECONF 用于指定文件 bblayers.conf.sample 和 local.conf.sample 所在的路径。source 指令用于初始化 Yocto 编译环境，这条指令会创建 build 目录，将 bblayers.conf.sample 和 local.conf.sample 复制到 build/conf 目录，并重命名为 bblayers.conf 和 local.conf，注意脚本会将当前工作目录切换到 build。最后的 bitbake 指令启动编译。如果需要再次编译，直接执行最后一条指令即可。如果终端退出重新登录，或使用一个新的终端，所有指令都要执行。

```
cd path/to/codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64
source meta/poky/oe-init-build-env
bitbake mtk-core-image-auto8676
```

⁵ <https://docs.yoctoproject.org/ref-manual/system-requirements.html#supported-linux-distributions>

⁶ <https://docs.yoctoproject.org/ref-manual/system-requirements.html#required-packages-for-the-build-host>

1.3.3 软件构成

软件编译完成后，软件在 build/tmp/deploy/images/auto8676p1_64 目录中。表 1-9 列出了主要的镜像文件及其说明。

表 1-9. 镜像文件说明

文件名	分区	说明
MT6897_Android_scatter.xml	N/A	分区表，由 ptgen-v2.bb 编译出来。
download_agent/DA_BR.bin	N/A	download agent，用于烧录。由 collect.bb 编译出来。
bl2.img	preloader_a preloader_b	bootloader，由 lk2.bb 编译出来。
modem.img	modem_a	modem 固件，由 modem.bb 编译出来。
spmfw.img	spmfw_a	System Power Management 固件，由 collect.bb 编译出来。
mcf_ota.img	mcf_ota_a	由 collect.bb 编译出来。
audio_dsp.img	audio_dsp_a	audio DSP 固件，由 collect.bb 编译出来。
pi_img.img	pi_img_a	由 collect.bb 编译出来。
dpm.img	dpm_a	由 collect.bb 编译出来。
scp-fit.img	scp_a	System Companion Processor 固件，由 tinysys-scp.bb 编译出来。
ccu.img	ccu_a	Camera Control Unit 固件，由 collect.bb 编译出来。
vcp.img	vcp_a	vcp firmware，由 tinysys-vcp.bb 编译出来。
sspm.img	sspm_a	Secure System Power Manager 固件，由 collect.bb 编译出来。
mcupm.img	mcupm_a	power/performance manager 固件，由 collect.bb 编译出来。
gpueb.img	gpueb_a	GPU firmware，由 tinysys-gpueb.bb 编译出来。
apusys.img	apusys_a	APU firmware，由 tinysys-apusys.bb 编译出来。
boot.img	boot_a	Linux kernel，fit 格式，包含 kernel img、dtb 和 initramfs。由 linux-mtk-extension_6.1.bb 编译出来。其中 initramfs 由 core-image-minimal-initramfs.bb 编译出来。
tee.img	tee_a	ATF，由 collect.bb 编译出来。
connsys_bt.img	connsys_bt_a	保留分区，实际未使用。由 collect.bb 编译出来。
connsys_wifi.img	connsys_wifi_a	保留分区，实际未使用。由 collect.bb 编译出来。
connsys_gnss.img	connsys_gnss_a	GNSS 固件，由 collect.bb 编译出来。

文件名	分区	说明
logo.img	logo_a	logo, 由 makelogo.bb 编译出来。
system.ext4	system	根文件系统, 由 mtk-core-image-auto8676.bb 编译出来。
userdata.ext4	userdata	/data 分区, 由 mkusrdata.bbclass 编译出来。

1.3.4 烧录软件

烧录软件需要用到 Type-C 线和 SP_Flash_Tool 工具。如果没有 SP_Flash_Tool 工具, 请访问 [Online](#)⁷ 网站下载。打开网页后, 搜索 SP_Flash_Tool, 下载最新版本, 如图 1-4 所示。下载完成后解压下载的压缩包。



图 1-4. 下载 SP_Flash_Tool

首先用 Type-C 线将平台与电脑连接起来。然后双击 SP_Flash_Tool_V6/SPFlashToolV6.exe 打开烧录工具。第一步选择 Download-XML 文件, 点击右上角的 choose 按钮, 选择软件包下面的 download_agent/flash.xml 文件。第二步在 Download 按钮下方的下拉框中选择 Format All + Download。效果如图 1-5 所示。第三步, 确保平台处于断电状态。第四步, 点击 Download 按钮。第五步, 按住 download key (KPCOL0, SW907) 不松手, 平台上电, 这时会自动开始烧录, 如图 1-6 所示。此时可以松开按键。

如果有遇到错误弹窗, 请先将平台断电, 点击 Stop 按钮退出下载模式。确保平台掉电完毕后, 再尝试烧录。

⁷ <https://online.mediatek.com/apps/tool/>

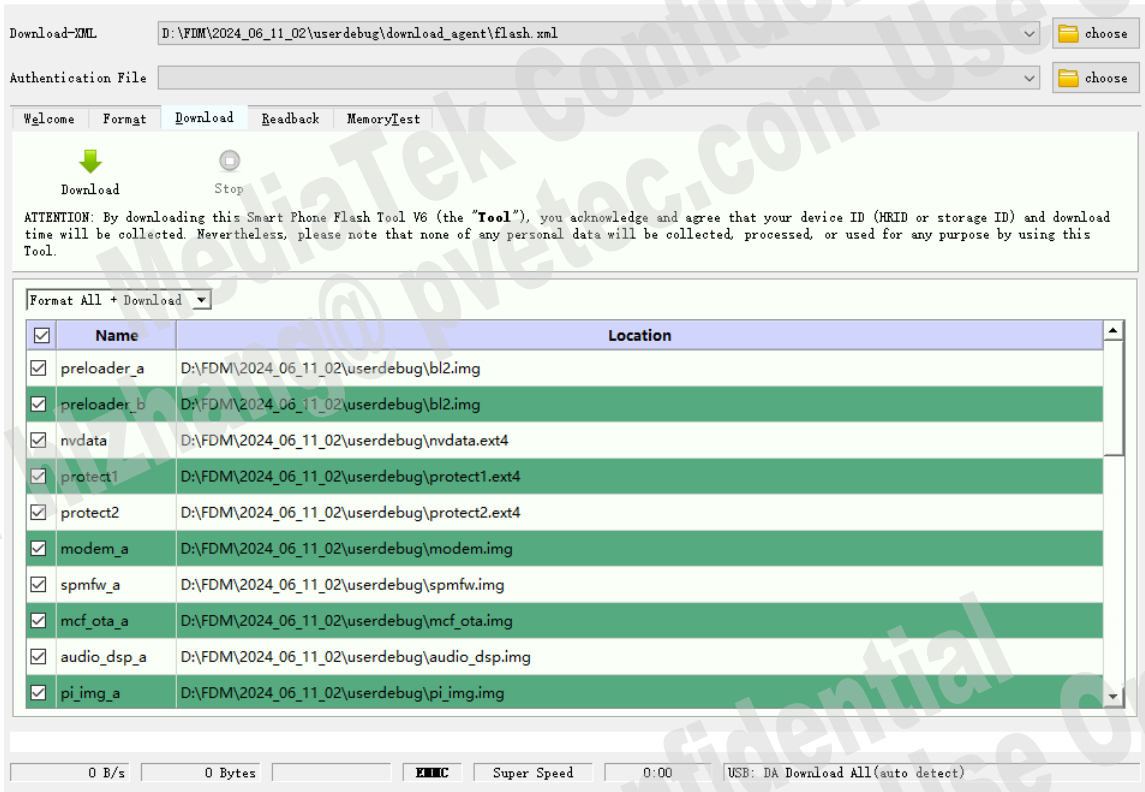


图1-5. 加载 flash.xml

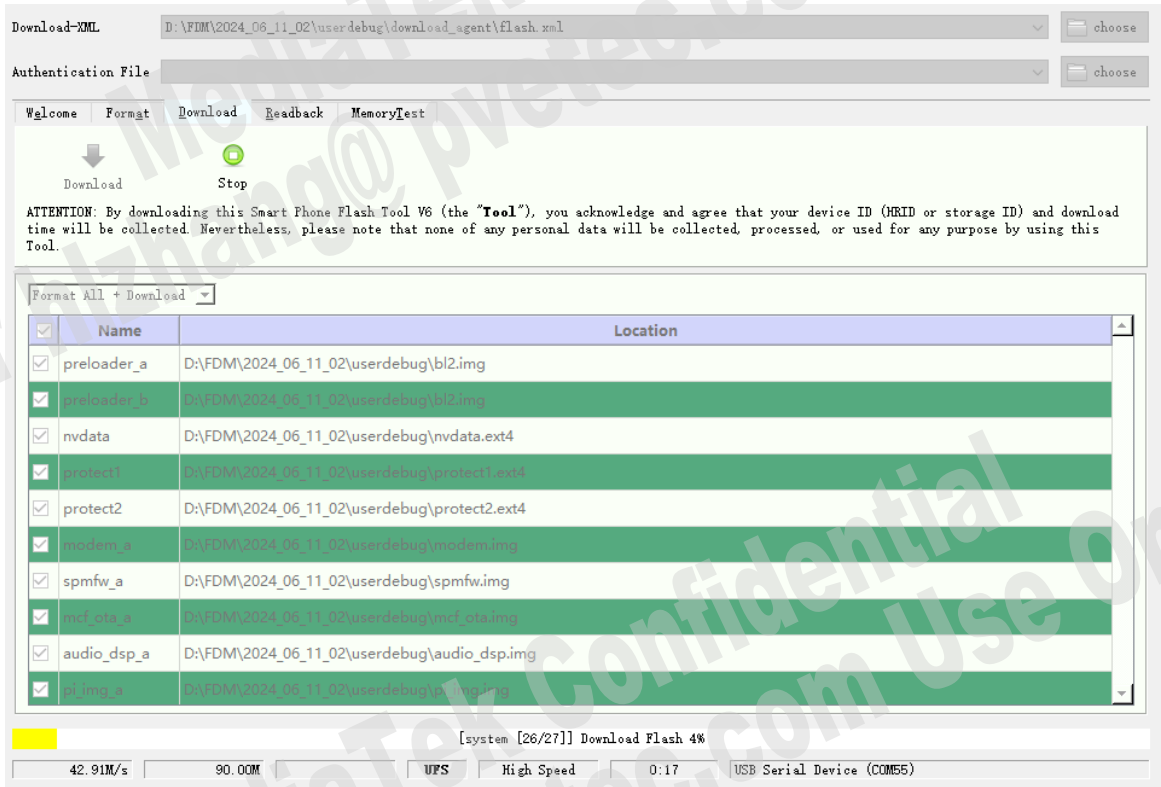


图 1-6. 软件烧录

1.3.5 编译 SDK

使用如下指令编译 SDK。与编译软件相比，bitbake 指令多了 `-c populate_sdk` 选项。

```
cd path/to/codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64
source meta/poky/oe-init-build-env
bitbake mtk-core-image-auto8676 -c populate_sdk
```

1.3.6 使用 SDK

SDK 编译成功后，安装包在 `build/tmp/deploy/sdk` 目录。对于 MT8676 Yocto5.0 来说，安装包的名字是 `poky-glibc-x86_64-mtk-core-image-auto8676-aarch64-auto8676p1_64-toolchain-5.0.sh`。

安装过程如图 1-7 所示，首先清空环境变量 `LD_LIBRARY_PATH`，如果不清空会导致 SDK 设置失败。第二步启动安装脚本，输入 SDK 的安装目录，输入 y 确认安装，等待安装完成。按照安装脚本的提示，执行设置脚本，现在就可以使用 SDK 安装好的工具链了。

```
$ unset LD_LIBRARY_PATH
$ ./poky-glibc-x86_64-mtk-core-image-auto8676-aarch64-auto8676p1_64-toolchain-5.0.sh
Poky (Yocto Project Reference Distro) SDK installer version 5.0
=====
Enter target directory for SDK (default: /opt/poky/5.0): /worktmp/codebase/mt8676sdk
You are about to install the SDK to "/worktmp/codebase/mt8676sdk". Proceed [Y/n]? y
Extracting SDK.....done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
$ . /worktmp/codebase/mt8676sdk/environment-setup-aarch64-poky-linux
$ . /worktmp/codebase/mt8676sdk/environment-setup-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
$ . /worktmp/codebase/mt8676sdk/environment-setup-aarch64-poky-linux
$ aarch64-poky-linux-gcc --version
aarch64-poky-linux-gcc (GCC) 13.2.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
mtk19899@mszsdclx1033:~$
```

图 1-7. 安装 SDK

SDK 安装目录结构如图 1-8 所示，根目录包含环境设置脚本、版本信息等文件。sysroots/aarch64-poky-linux 目录包含 MT8676 Yocto 编译出来的所有可执行文件、头文件、动态库、静态库等文件，方便第三方在非 Yocto 环境下进行开发。这个目录下面的 ELF 文件是 aarch64 格式。sysroots/x86_64-pokysdk-linux 目录包含一些本地工具，例如 aarch64 工具链放在 `sysroots/x86_64-pokysdk-linux/usr/bin/aarch64-poky-linux` 目录。还有很多其他版本的工具链可以使用，例如 `aarch64-pokymllib32-linux`、`arm-poky-linux` 等。

```
.
├── environment-setup-aarch64-poky-linux
├── environment-setup-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
├── site-config-aarch64-poky-linux
├── site-config-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
├── sysroots
│   ├── aarch64-poky-linux
│   └── x86_64-pokysdk-linux
│       ├── bin
│       ├── environment-setup.d
│       ├── etc
│       ├── lib
│       ├── sbin
│       └── usr
│           ├── bin
│           ├── .....
│           ├── aarch64-poky-linux
│           ├── aarch64-poky-linux-musl
│           ├── aarch64-pokymllib32-linux
│           ├── aarch64-pokymllib32-linux-musl
│           ├── arm-poky-linux
│           ├── arm-poky-linux-gnueabi
│           ├── arm-poky-linux-musl
│           ├── arm-poky-linux-musleabi
│           ├── arm-pokymllib32-linux
│           ├── arm-pokymllib32-linux-gnueabi
│           ├── arm-pokymllib32-linux-musl
│           ├── arm-pokymllib32-linux-musleabi
│           └── .....
├── version-aarch64-poky-linux
└── version-cortexa7hf-neon-vfpv4-pokymllib32-linux-gnueabi
```

图 1-8. 目录结构

1.4 Yocto 客制化

1.4.1 常用变量

表 1-10 列出了一些常用变量，这些变量在编写 bb 的时候会经常用到。Yocto 的变量非常多，要玩转 Yocto 仅看表 1-10 是远远不够的。如果需要查看所有变量的值，请参考 1.6.3 章节。

表 1-10. Yocto 常用变量

变量名	说明
MACHINE	指定目标设备。一般在 local.conf 定义。
MTK_PROJECT	指定项目名字。一般在 <machine>.conf 定义。
PN	包名，即 pacakge name。一般从文件名解析出来，以 foo_2.6.bb 为例，PN=foo。
PV	版本号，即 package version。一般从文件名解析出来，以 foo_2.6.bb 为例，PV=2.6。如果文件名没有指定版本号，例如 bar.bb，则 PV 取默认值 1.0。也可以在 bb 中直接设定 PV。
SRC_URI	源代码 URI。支持 file、http、https、git 等协议。
WORKDIR	bb 工作目录。bb 所有的任务都在工作目录中完成。

变量名	说明
S	源代码目录。do_fetch 任务会将 SRC_URI 变量指定的源代码下载到 S 目录。
B	编译目录。do_compile 任务会在此目录中工作。
D	安装目录。do_install 任务会将编译之后的产物安装到此目录。

1.4.2 bb 文件的基本结构

bb 文件是 Bitbake 配方文件（recipe），bb 文件描述了如何构建特定软件包，包括构建依赖、从哪里获取源代码、如何打补丁、如何编译和安装软件以及如何打包。bb 文件由一系列的变量定义和可执行任务组成。关于 bb 文件的语法和运算符，请参考 [Bitbake 文档](#)⁸。

[meta-skeleton](#)⁹提供了若干个示例 bb，以 [hello 1.0.bb](#)¹⁰为例，介绍 bb 文件的基本结构。首先，定义了软件包的一些基本信息，例如描述、分组、license 以及 license 文件的校验和。这些信息通过一些标准变量告诉 Bitbake 构建引擎。

```
DESCRIPTION = "Simple helloworld application"
SECTION = "examples"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"
```

通过 SRC_URI 变量指定从哪里下载源代码，通过 S 变量指定源代码的位置。Bitbake 支持多种协议下载源代码，例如 file、git、http、https、ssh、s3 等等。这里使用了 file 协议。对于 file 协议，Bitbake 会在变量 FILESEXTRAPATHS 指定的路径中寻找文件。与 PATH 环境变量类似，FILESEXTRAPATHS 变量由若干个以冒号分隔的路径组成。

```
SRC_URI = "file://helloworld.c"

S = "${WORKDIR}/sources"
UNPACKDIR = "${S}"
```

do_compile 任务用于编译软件，Bitbake 默认提供了一个空的实现，所以需要重写此任务。Bitbake 任务的名字必须以 “do_” 开头，支持 shell 任务或 python 任务。这里定义的 do_compile 是 shell 任务。在 shell 任务中，可以通过 “\${VAR_NAME}” 的形式引用 bb 文件定义的变量。

```
do_compile() {
    ${CC} ${LDFLAGS} helloworld.c -o helloworld
}
```

⁸ <https://docs.yoctoproject.org/bitbake/2.8/bitbake-user-manual/bitbake-user-manual-metadata.html>
⁹ <https://git.yoctoproject.org/poky/tree/meta-skeleton>
¹⁰ https://git.yoctoproject.org/poky/tree/meta-skeleton/recipes-skeleton/hello-single/hello_1.0.bb

do_install 任务用于安装软件，软件必须安装到 \${D} 目录下。Bitbake 定义了一系列用于安装不同文件的变量，例如可执行文件安装到\${bindir} 下，动态库安装到\${libdir}下，配置文件安装到\${sysconfdir} 下。

```
do_install() {
    install -d ${D}${bindir}
    install -m 0755 helloworld ${D}${bindir}
}
```

有些时候，软件包自己维护了一套配置、编译、安装脚本，不需要在 bb 文件重新实现一套了。例如对使用 autotools 的软件包来说，可以使用标准命令 “./configure && make && make install” 来完成配置、编译和安装。对于这些软件包，不需要从零开始写 do_compile 或 do_install，直接使用 Yocto 写好的即可。以 [hello_2.10.bb](#)¹¹为例，只需要继承 autotools-brokensep 类，就可以编译使用 autotools 的软件包，极大的减轻了编写 bb 的负担。

```
DESCRIPTION = "GNU Helloworld application"
SECTION = "examples"
LICENSE = "GPL-3.0-only"
LIC_FILES_CHKSUM = "file://COPYING;md5=d32239bcb673463ab874e80d47fae504"

SRC_URI = "${GNU_MIRROR}/hello/hello-${PV}.tar.gz"
SRC_URI[sha256sum] = "31e066137a962676e89f69d1b65382de95a7ef7d914b8cb956f41ea72e0f516b"

inherit autotools-brokensep gettext
```

1.4.3 开机自启动

以 1.4.2 章节介绍的 hello_1.0.bb 为基础，介绍如何开机自动运行某个程序。MT8676 Yocto 使用 systemd 作为 1 号进程，所以首先需要写一个 service 文件。以下是 helloworld.service 示例。关于 systemd service 的更多信息，可以参考 [systemd 文档](#)¹²。

```
[Unit]
Description=hello world service
After=basic.target

[Service]
Type=oneshot
ExecStart=/usr/bin/helloworld

[Install]
WantedBy=multi-user.target
```

将 service 文件添加到 SRC_URI 变量中。

```
SRC_URI = "file://helloworld.c file://helloworld.service"
```

¹¹ https://git.yoctoproject.org/poky/tree/meta-skeleton/recipes-skeleton/hello-autotools/hello_2.10.bb

¹² <https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html>

由于 service 文件不需要编译，所以 do_compile 保持不变。在 do_install 任务，需要将 service 文件安装到 systemd 指定的目录中。

```
do_install() {
    install -d ${D}${bindir}
    install -m 0755 helloworld ${D}${bindir}
    install -d ${D}${systemd_unitdir}/system
    install helloworld.service ${D}${systemd_unitdir}/system
}
```

继承 systemd 类，指定 service 的名字，开启开机自启动。代码如下。至此，开机自启动配置完成。

```
inherit systemd
SYSTEMD_PACKAGES = "${PN}"
SYSTEMD_SERVICE:${PN} = "helloworld.service"
```

支持开机自启动的完整版 hello_1.0.bb 如下。

```
DESCRIPTION = "Simple helloworld application"
SECTION = "examples"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://helloworld.c file://helloworld.service"

S = "${WORKDIR}/sources"
UNPACKDIR = "${S}"

do_compile() {
    ${CC} ${LDFLAGS} helloworld.c -o helloworld
}

do_install() {
    install -d ${D}${bindir}
    install -m 0755 helloworld ${D}${bindir}
    install -d ${D}${systemd_unitdir}/system
    install helloworld.service ${D}${systemd_unitdir}/system
}

inherit systemd
SYSTEMD_PACKAGES = "${PN}"
SYSTEMD_SERVICE:${PN} = "helloworld.service"
```

1.4.4 依赖

构建软件包时，依赖是一个很令人头疼的问题。不过 Yocto 很好地解决了这个问题。假如 foo.bb 在编译时依赖 bar.bb 的产物，只需要在 foo.bb 添加如下代码，即把依赖的软件包追加到 DEPENDS 变量。Yocto 会保证 bar 先于 foo 编译，并且在编译 foo 时，会将 bar 安装到 \${D} 的文件复制到 foo 的工作目录（\${WORKDIR}）。这样编译 foo 时可以尽情的使用 bar 安装的文件，例如静态库、动态库、头文件等。

```
# foo.bb
DEPENDS += "bar"
```

如果 foo.bb 在运行时依赖 bar.bb，需要在 foo.bb 添加如下代码。注意变量 RDEPENDS 开头的 “R” 表示运行时，并且还多了一个 “:\${PN}”，这个不能少。

```
# foo.bb
RDEPENDS:${PN} += "bar"
```

1.4.5 树外驱动

编译树外驱动比较特殊，因为需要使用 kernel 和其他树外驱动编译出来的符号，同时自己编译出来的符号也要给其他树外驱动使用。考虑到 MT8676 的树外驱动比较多，为了避免出现较多的依赖问题，这里给出了一个模板。编译树外驱动需要 kernel 符号，所以依赖 virtual/kernel。

```
DESCRIPTION = "sample out of tree driver"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/GPL-2.0-only;md5=801f80980d171dd6425610833a22dbe6"
DEPENDS = "virtual/kernel"
```

编译树外驱动必须继承 module 类。这里使用了 fog 协议来获取源代码，关于 fog 协议的介绍请参考 [1.4.7 章节](#)。

kernel-toolchain.inc 用于设置工具链。

```
# for out-of-tree module bb, must inherit *module*
inherit fog module
SRC_URI = "fog://src/kernel/modules/module-name;name=module-name"
SRCREV_module-name = "${AUTOREV}"

require conf/common/kernel-toolchain.inc

S = "${WORKDIR}/git"
MODULE = "${S}"
LINUX_SRC = "${STAGING_KERNEL_DIR}"
MODULE_NAME = "gpio_sap_ctrl"
```

变量 EXTRA_SYMBOLS 用于指定额外的符号文件，MT8676 Linux Kernel 分为 common 和 device module 两部分，所以 EXTRA_SYMBOLS 默认添加了 device module 的符号文件。如果依赖其他树外驱动的符号，首先将模块名追加到 DEPENDS 变量，然后按照示例，把符号文件的路径追加到 EXTRA_SYMBOLS 变量。

```
# device_module symvers
EXTRA_SYMBOLS += "${TMPDIR}/work-shared/${MACHINE}/device-module-build-artifacts/Module.symvers"

# if you module need other out-of-tree module symbol, please append DEPENDS & EXTRA_SYMBOLS. below is sample.
# DEPENDS += "oot-module1 oot-module2"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module1/Module.symvers"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module2/Module.symvers"
```

如果 Makefile 文件需要更多的变量，可以通过 export 指令传递给 Makefile。同时，不需要自己编写 do_compile 任务和 do_install 任务。module 类会处理好。

```
# export Makefile required variables. If need more, please add here.
export M="${MODULE}"
export EXTRA_SYMBOLS
export KBUILD_EXTRA_SYMBOLS

# no need write self do_compile / do_install function
# module.bbclass will handle it
```

完整的模板如下：

```

DESCRIPTION = "sample out of tree driver"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/GPL-2.0-only;md5=801f80980d171dd642561083
3a22dbe6"
DEPENDS = "virtual/kernel"

# for out-of-tree module bb, must inherit *module*
inherit fog module
SRC_URI = "fog://src/kernel/modules/module-name;name=module-name"
SRCREV_module-name = "${AUTOREV}"

require conf/common/kernel-toolchain.inc

S = "${WORKDIR}/git"
MODULE = "${S}"
LINUX_SRC = "${STAGING_KERNEL_DIR}"
MODULE_NAME = "gpio_sap_ctrl"

# device_module symvers
EXTRA_SYMBOLS += "${TMPDIR}/work-shared/${MACHINE}/device-module-build-artifacts/Module.symvers"

# if you moudle need other out-of-tree module symbol, please append DEPENDS & EXTRA_SYMBOL
S. below is sample.
# DEPENDS += "oot-module1 oot-module2"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module1/Module.symvers"
# EXTRA_SYMBOLS += "${RECIPE_SYSROOT}${includedir}/oot-module2/Module.symvers"

INSANE_SKIP:${PN} += "already-stripped"
INSANE_SKIP:${PN} += "installed-vs-shipped"

# append EXTRA_SYMBOLS to KBUILD_EXTRA_SYMBOLS
python __anonymous () {
    d.appendVar('KBUILD_EXTRA_SYMBOLS', " " + d.getVar('EXTRA_SYMBOLS'))
}

# export Makefile required variables. If need more, please add here.
export M="${MODULE}"
export EXTRA_SYMBOLS
export KBUILD_EXTRA_SYMBOLS

# no need write self do_compile / do_install function
# module.bbclass will handle it

```

1.4.6 image bb

image bb 用于生成根文件系统。以 [core-image-minimal.bb](https://git.yoctoproject.org/poky/tree/meta/recipes-core/images/core-image-minimal.bb)¹³ 为例介绍。image bb 必须继承 core-image 类。通过 IMAGE_INSTALL 变量指定根文件系统需要安装的软件包，如果想安装更多的软件包到根文件系统，只需要追加软件包到 IMAGE_INSTALL 变量即可。MT8676 的 image bb 是 mtk-core-image-auto8676.bb。

```
SUMMARY = "A small image just capable of allowing a device to boot."

IMAGE_INSTALL = "packagegroup-core-boot ${CORE_IMAGE_EXTRA_INSTALL}"

IMAGE_LINGUAS = " "

LICENSE = "MIT"

inherit core-image

IMAGE_ROOTFS_SIZE ?= "8192"
IMAGE_ROOTFS_EXTRA_SPACE:append = "${@bb.utils.contains("DISTRO_FEATURES", "systemd", " + 4096", "", d)}"
```

根文件系统镜像的文件格式在<machine>.conf 指定。以 auto8676p1_64.conf 为例，根文件系统镜像的格式是 ext4。

```
# auto8676p1_64.conf
IMAGE_FSTYPES ?= "ext4"
```

1.4.7 fog 简介

fog 是 “file or git” 的缩写，是一个 MediaTek 扩展的源代码下载协议。fog 是以 git 仓库为单位，将整个仓库抓到 \${WORKDIR}。fog://协议会根据一些规则将源代码的下载协议转换为 file://或 git://协议。

fog 示例如下。首先要继承 fog 类，然后将 fog://协议格式的源代码路径追加到 SRC_URI 变量，name 参数是必须设置的，用于设置模块的名字。SRC_REV 变量用于指定模块的版本，一般设置为 \${AUTOREV}。最后设置 S 变量，必须设置为 “\${WORKDIR}/git” 或 git 目录的子目录，取决于编译入口在哪个目录。

```
inherit fog
SRC_URI = "fog://src/path;name=module-name"
SRCREV_module-name = "${AUTOREV}"

S = "${WORKDIR}/git"
```

fog 支持多个 git 仓库，示例如下。与单个 git 仓库相比，必须指定 destsuffix 参数，用于指定仓库所在的子目录。SRC_URI 也要设置多个。多了 SRCREV_FORMAT，用于设置 SRCREV 格式。

¹³ <https://git.yoctoproject.org/poky/tree/meta/recipes-core/images/core-image-minimal.bb>

```
inherit fog
SRC_URI="fog://src/path;name=module-name1;destsuffix=git/sub-dir1"
SRC_URI="fog://src/path;name=module-name2;destsuffix=git/sub-dir2"
SRC_URI="fog://src/path;name=module-name3;destsuffix=git/sub-dir3"
SRCREV_module-name1 = "${AUTOREV}"
SRCREV_module-name2 = "${AUTOREV}"
SRCREV_module-name3 = "${AUTOREV}"
SRCREV_FORMAT="module-name1_module-name2_module-name3"

S = "${WORKDIR}/git"
```

1.4.8 创建 project

表 1-11 列出了创建一个 project 需要新建的文件，建议从 auto8676p1_64 复制，然后修改相关文件。也可以使用一键创建脚本 meta/meta-mediatek-mt8676/scripts/create_project.sh，按照提示操作即可。

表 1-11. 创建 project

项目	示例
machine.conf	meta/meta-mediatek-mt8676/conf/machine/auto8676p1_64.conf
templates	meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64/bblayers.conf.sample meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64/local.conf.sample
dts	src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/auto8676p1_64.dts
defconfig	src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/configs/auto8676p1_64_defconfig
分区表	meta/meta-mediatek-mt8676/recipes-bsp/ptgen/files/ auto8676p1_64/partition_table.csv
ko table	meta/meta-mediatek-mt8676/recipes- kernel/linux/ko_order_table/auto8676p1_64/ko_order_table.csv
fstab	meta/meta-mediatek-mt8676/recipes-core/base-files/base-files/auto8676p1_64/fstab
collect-bins	prebuilt/bsp/collect-bins/mt8676/auto8676p1_64
dws	src/devtools/dct/dws/mt6897/auto8676p1_64.dws
lk2	src/bsp/lk2/project/auto8676p1_64.mk
scp	src/tinysys/mt8676/scp/project/RV55_A/mt6897/auto8676p1_64

1.4.9 创建 meta layer

Yocto meta layer 本质上是一个包含 conf/layer.conf 文件的目录，目录名字建议以 “meta-” 开头，例如 meta-foo、meta-bar 等等。所以创建 meta layer 首先需要创建 conf/layer.conf 文件。以创建 meta-foo 为例，示例 layer.conf 如下。BBPATH 将当前 layer 路径添加到 Bitbake 搜索路径中，BBFILES 为指定 bb 文件所在的路径，BBFILE_COLLECTIONS 为当前 layer 指定一个唯一的标识符，BBFILE_PATTERN 设置 bb 文件的目录前缀，

BBFILE_PRIORITY 设置 bb 文件的优先级，数字越大优先级越高。如果两个 layer 有相同名字的 bb，则使用优先级高的。LAYERSERIES_COMPAT 用于设置 layer 支持的 Yocto 版本。

```
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":{LAYERDIR}"

# We have a recipes-* directories, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
           ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "foo"
BBFILE_PATTERN_foo = "^${LAYERDIR}/"
BBFILE_PRIORITY_foo = "10"

LAYERSERIES_COMPAT_foo = "scarthgap"
```

图 1-9 展示了 meta layer 的目录结构。除 conf/layer.conf 文件外，classes 目录用于存放 bbclass，conf/machine 和 conf/templates 目录用于存放 project 相关的文件。“recipes-”开头的目录用于存放 bb 和 bbappend 文件，这些目录的结构取决于 BBFILES 变量，一般设置为 “\${LAYERDIR}/recipes-*/*/*.bb \${LAYERDIR}/recipes-*/*/*.bbappend”。

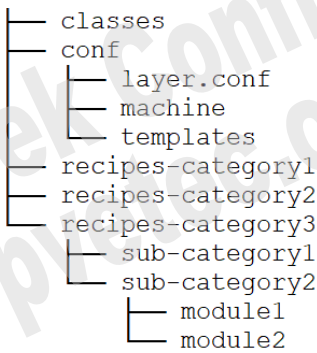


图 1-9. meta layer 目录结构

1.5 Yocto Build Mode

Yocto 没有定义 build mode，默认只能编译出一种软件。但是在项目周期中，需要在不同的阶段使用不同的软件，MediaTek 针对 DEV/SQC/MP 提出了类似 Android 的解决方案。表 1-12 列出了不同 build mode 之间的差异。

表 1-12. Yocto build mode

Build Mode	Android Config Setting	Yocto Config Setting	Usage
user	1. Disable debug configs (TFA/LK/Preloader/Kernel/TEE/Fwk/APK/....) 2, Disable debug features (AEE/UART/ADB/...)	1. Disable debug configs (TFA/LK2.0/Kernel/TEE/APP/....) 2. Disable debug features (AEE/UART/ADB/...)	1. Products MP 2. SQC performance test

Build Mode	Android Config Setting	Yocto Config Setting	Usage
userdebug	1. Disable debug configs (TFA/LK/Preloader/Kernel/TEE/Fwk/APK/....) 2. Enable debug features (AEE/UART/ADB/...)	1. Disable debug configs (TFA/LK2.0/Kernel/TEE/APP/....) 2. Enable debug features (AEE/UART/ADB/...)	1. SQC main test 2. RD DEV/UT/IT test
eng	1. Enable debug configs (TFA/LK/Preloader/Kernel/TEE/Fwk/APK/....) 2. Enable debug features (AEE/UART/ADB/...)	1. Enable debug configs (TFA/LK2.0/Kernel/TEE/APP/....) 2. Enable debug features (AEE/UART/ADB/...)	1. RD debug

1.5.1 使用方法

MediaTek Yocto 使用环境变量 TARGET_BUILD_VARIANT 来区分三种 build mode。环境变量的值可以设置为 user、userdebug 或 eng，分别用于编译出 user 软件、userdebug 软件、eng 软件。如果没有设置环境变量，或者值不是 user、userdebug、eng 中的一个，那么会编译出 userdebug 软件。

为了在 bb 文件中区分这三种模式，MediaTek 基于环境变量 TARGET_BUILD_VARIANT 的值，向变量 DISTRO_FEATURES 和 OVERRIDES 追加不同的值，如表 1-13 所示。

表1-13. Build variant

TARGET_BUILD_VARIANT	DISTRO_FEATURES	OVERRIRES
user	variant-user	variant-user
userdebug	variant-userdebug	variant-userdebug
eng	variant-eng	variant-eng

以下是编译 eng 软件的指令，如果需要编译 user 或 userdebug 软件，只需要更改环境变量 TARGET_BUILD_VARIANT 的值即可。

```
cd path/to/codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676/conf/templates/auto8676p1_64
export TARGET_BUILD_VARIANT=eng
source meta/poky/oe-init-build-env
bitbake mtk-core-image-auto8676
```

1.5.2 适配

模块需要对 build mode 进行适配，才能编译出真正的 user、userdebug 或 eng 软件。首先需要定义这三种模式的区别，然后针对性的修改 bb 文件去适配。这里展示两个例子，介绍如何适配。

以 linux kernel (linux-mtk-extension_6.1.bb) 为例, linux kernel 使用 defconfig 进行配置。user 软件只需要使用 project defconfig 文件, userdebug 软件使用 project defconfig 和 userdebug.config 文件, eng 软件使用 project defconfig 和 eng.config 文件。相关适配代码如下。变量 VARIANT_CONFIG 定义了额外的 config 文件, 默认是空, 如果使用 userdebug, 则设置为 userdebug.config, eng 类似。这里使用了 Yocto 的条件变量语法。最后在 do_configure 任务之前 (prepend) 追加一段代码, 使用脚本 merge_config.sh 合并两个 config 文件。

```
VARIANT_CONFIG = ""
VARIANT_CONFIG:variant-eng = "${S}/${KCONFIG_EXT_PREFIX}/kernel/configs/eng.config"
VARIANT_CONFIG:variant-userdebug =
"${S}/${KCONFIG_EXT_PREFIX}/kernel/configs/userdebug.config"

do_configure:prepend() {
    bbnote "VARIANT_CONFIG is ${VARIANT_CONFIG}"

    install -d ${B}/arch/${KERNEL_ARCH}/configs/
    KCONFIG_CONFIG=${B}/arch/${KERNEL_ARCH}/configs/${KBUILD_CONFIG} \
        ${S}/scripts/kconfig/merge_config.sh -m \
        -r ${MODULEDIR}/arch/${KERNEL_ARCH}/configs/${KBUILD_CONFIG} \
        ${VARIANT_CONFIG}
    ...
}
```

以 adb (android_tools_5.1.1.r37.bbappend) 为例。user 软件需要关闭 adb, userdebug 和 eng 软件打开 adb。相关适配代码如下。这里借用 Yocto 条件变量语法, 仅在 user 软件时, 将 SYSTEMD_SERVICES 和 FILES 变量清空, 并向 do_install 任务追加删除相关文件的指令。

```
# variant user disable adbd
SYSTEMD_SERVICE:${PN}-adbd:variant-user = ""
FILES:${PN}-adbd:variant-user = ""
do_install:append:variant-user() {
    rm -rfv "${D}${bindir}/adbd"
    rm -rfv "${D}${systemd_unitdir}/system/android-tools-adbd.service"
}

# avoid build fail
INSANE_SKIP:${PN} += "installed-vs-shipped"
```

1.6 Yocto 调试技巧

1.6.1 build 目录结构

软件编译完成后, 目录结构如图 1-10 所示。与原始的目录结构相比, 多了 build、downloads、sstate-cache 目录。build 目录由变量 TOPDIR 指定, 包含编译中间文件、镜像、rpm 包等文件。关于 downloads 目录和 sstate-cache, 请参考 1.6.4 章节。

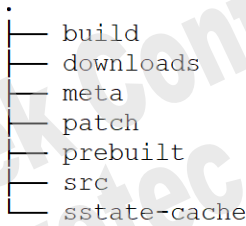


图 1-10. 编译完成后的目录结构

build 目录包含非常多的信息，熟悉 build 目录有助于调试 Yocto。变量 TOPDIR 指向 build 目录。表 1-14 列出了 build 目录常用的文件和目录。

表 1-14. build 目录常用文件和目录

文件或目录	说明
build/conf/local.conf	从 local.conf.sample 复制而来。
build/conf/bblayers.conf	从 bblayers.conf.sample 复制而来。
build/conf/templateconf.cfg	包含环境变量 TEMPLATECONF 的值。
build/tmp/deploy/images/<project>	包含用于烧录的镜像文件。
build/tmp/deploy/rpm	包含 bb 编译之后生成的 rpm 包。
build/tmp/deploy/license	子目录的格式是<PN>，包含对应 bb 的 license 信息。
build/tmp/sysroots-components	子目录的格式是<ARCH>/<PN>，包含对应 bb 安装到 rootfs 的文件。
build/tmp/work	子目录的格式是<ARCH>/<PN>/<PV>，对应 bb 的工作目录。bb 的所有任务都是在工作目录完成的。具体请参考 1.6.2 章节。
build/tmp/work-shared	共享的工作目录。如果 bb 之间需要共享文件，可以将文件放在此目录中。

1.6.2 工作目录常用子目录

工作目录在 build/tmp/work 下，bb 的所有任务都是在工作目录完成的，变量 WORKDIR 指向 bb 的工作目录。表 1-15 列出了工作目录常用子目录。借助工作目录下的文件，可以快速厘清问题。例如 do_compile 执行失败，可以检查 temp/run.do_compile 和 log.do_compile 文件；例如确认文件被打包到哪一个软件包，可以检查 packages-split 目录。

表 1-15. 工作目录常用子目录

文件或目录	说明
git	存放 SRC_URI 变量指定的源代码。也有 bb 将源代码放在\${PN}-\${PV}子目录。S 变量指向此目录。
build	存放编译中间文件和最终文件的目录。B 变量指向此目录。有时候 B 变量等于 S 变量。
recipe-sysroot	存放 DEPENDS 变量指定的软件包的文件。编译时会在此目录下寻找头文件和库。
recipe-sysroot-native	与 recipe-sysroot 变量类似，存放 DEPENDS 变量指定的 “*-native” 软件包的文件，编译时会用到的本地工具，例如交叉编译器。
image	存放 do_install 任务安装的文件，D 变量指向此目录。
package	存放最终打包的文件。
packages-split	与 package 目录类似，但会按照包名划分子目录。一般来说，除默认的\${PN}包，Yocto 默认还会添加\${PN}-dbg、\${PN}-dev、\${PN}-doc 等几个包。
sysroot-destdir	此目录的文件是 image 目录的子集，如果被其他 bb 依赖，则此目录的文件会复制到对方 bb 的 recipe-sysroot 或 recipe-sysroot-native 目录。
temp	包含任务脚本和运行日志。以 do_compile 任务为例，run.do_compile 是任务脚本，log.do_compile 是任务日志。log.task_order 文件记录了任务的运行时间和运行顺序。

1.6.3 打印变量值

Yocto 的变量非常多，而且 yocto 的变量运算符非常的灵活，同时配置文件非常的多，这导致很难确定一个变量的最终取值。我们可以借用 bitbake 的 “-e” 选项，将所有变量的变化过程以及最终的值打印出来，可以极大的提高调试的效率。以 systemd 的 WORKDIR 变量为例。执行如下命令，将 systemd 的环境变量保存到 systemd.env 文件中。因为变量太多了，重定向保存到文件便于分析。

```
bitbake systemd -e > systemd.env
```

打开 systemd.env 文件，可以搜索到如下内容。可以看到，WORKDIR 变量总共有 2 次操作，第一次是在 bitbake.conf 的第 407 行设置变量值，第二次是在 documentation.conf 的第 467 行设置标签 doc 的值。最后打印了变量 WORKDIR 的值，借助这个方法，可以很轻松的定位哪一个配置文件对变量做了什么操作。

```
#
# $WORKDIR [2 operations]
#   set /mt8676/codebase/meta/poky/meta/conf/bitbake.conf:407
#     "${BASE_WORKDIR}/${MULTIMACH_TARGET_SYS}/${PN}/${PV}"
#   set /mt8676/codebase/meta/poky/meta/conf/documentation.conf:467
#     [doc] "The pathname of the working directory in which the OpenEmbedded build system
# builds a recipe. This directory is located within the TMPDIR directory structure and changes
# as different packages are built."
# pre-expansion value:
#   "${BASE_WORKDIR}/${MULTIMACH_TARGET_SYS}/${PN}/${PV}"
WORKDIR="/mt8676/codebase/build/tmp/work/aarch64-poky-linux/systemd/255.4"
```

1.6.4 加速 Yocto 编译

Yocto 从零开始编译非常的慢，主要慢在两个地方。第一，Yocto 会下载非常多的源码包和代码仓库。第二，很多没有修改的软件包也会从零开始编译。可以保存 Yocto 编译完成后生成的中间文件，来加速下一次编译。Yocto 编译完成后，会生成 downloads 和 sstate-cache 目录。downloads 目录包含下载的源码包和代码仓库，由变量 DL_DIR 指定；sstate-cache 目录包含编译缓存，由变量 SSTATE_DIR 指定。

在第一次编译之前，创建 build/conf/site.conf 文件，添加如下代码。这句代码会将下载的代码仓库打包成一个压缩包。编译完成后，将 downloads 和 sstate-cache 目录复制保存一份。例如分别保存到

"/path/to/your/saved/downloads/" 目录和 "/path/to/your/saved/sstate-cache/" 目录。

```
BB_GENERATE_MIRROR_TARBALLS = "1"
```

第二次编译之前，创建 build/conf/site.conf 文件，添加如下代码，SOURCE_MIRROR_URL 指向保存的 downloads 目录，SSTATE_MIRRORS 指向保存的 sstate-cache 目录（注意最后有一个固定的 PATH）。如果 bb 文件没有修改，Yocto 会直接从 SSTATE_MIRRORS 目录拿上一次编译好的结果。如果 sstate cache 没有命中，要下载软件包编译，Yocto 会优先从 SOURCE_MIRROR_URL 获取软件包。

```
SOURCE_MIRROR_URL ?= "file:///path/to/your/saved/downloads/"
INHERIT += "own-mirrors"
BB_GENERATE_MIRROR_TARBALLS = "1"

SSTATE_MIRRORS = "file://.* file:///path/to/your/saved/sstate-cache/PATH"
BB_SIGNATURE_HANDLER = "OEBasicHash"
BB_HASHSERVE = ""
```

需要注意的是，bb 修改之后，sstate cache 会失效，所以 sstate cache 要定期更新。downloads 目录一般不会失效。如果不想使用 sstate cache，可以给 bitbake 添加 "--no-setsccache" 选项。

1.7 开机流程

MT8676 Yocto 开机流程如图 1-11 所示。主要由 Bootloader、kernel、initramfs 和 systemd 等 4 大部分组成。

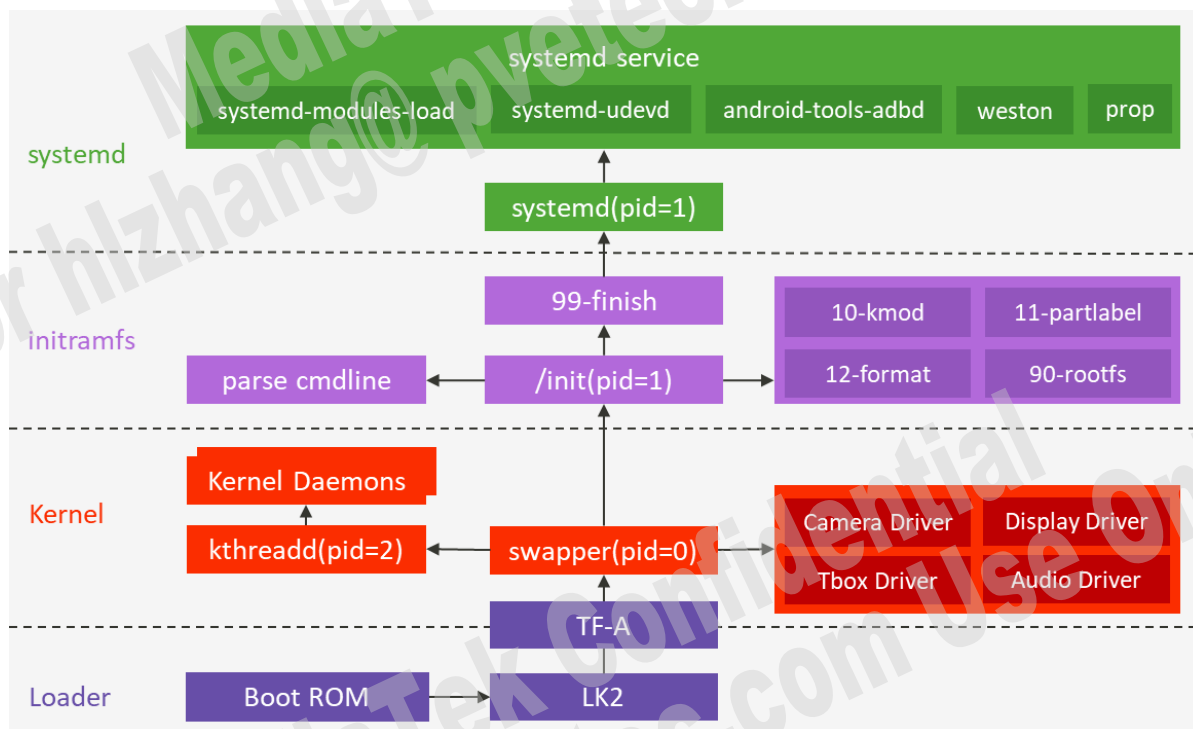


图 1-11. 开机流程

1.7.1 BootROM

BootROM 是一个硬件设备（类似于 SRAM/DRAM），里面存在着一段出厂自带的不可修改的已经固化的程序。上电之后，BootROM 开始执行内部事先已经设定好的 bootcode，加载引导程序 LK2 到 SRAM 中然后执行。主要功能如下：

- 引导加载 LK2 到 SRAM 里面执行。
- 当检测不到有效的镜像或者代码时，BootROM 引导进入下载模式。
- 执行一些校验工作。
- 硬件初始化，串口，flash 等。

1.7.2 LK2

LK2 即 bootloader。LK2 运行在 SRAM（SRAM 不需要初始化）中。LK2 的主要功能如下：

- 初始化 DRAM，以及各种外设。
- 加载 TF-A 并跳转到 TF-A 做初始化。
- 加载 kernel。
- 加载 tinysys（例如 gpueb、scp、vcp 等）。
- 跳转到 kernel。

1.7.3 Kernel

内核（Kernel）是操作系统的核心组件，它负责管理计算机硬件和软件资源，提供操作系统和应用程序的基本服务。内核是操作系统的第一个加载，并作为整个操作系统的核心运行在内存中，控制着所有的系统资源，如 CPU，内存，输入/输出，文件系统等。内核提供了底层的服务和接口，供应用程序进行调用和使用。

Kernel 汇编阶段（由 ENTRY(_stext)开始）：

1. 设置为 SVC 模式，关闭所有中断。
2. 获取 CPU ID。
3. 验证 dtb。
4. 创建页表项。
5. 配置 r13 寄存器，也就是设置打开 MMU 之后要跳转到的函数。
6. 使能 MMU。
7. 跳转到 start_kernel，进入 C 阶段。

Kernel C 阶段（由 kernel_init 开始），主要完成硬件平台相关的初始化工作，在相关初始化结束之后执行 init 程序。

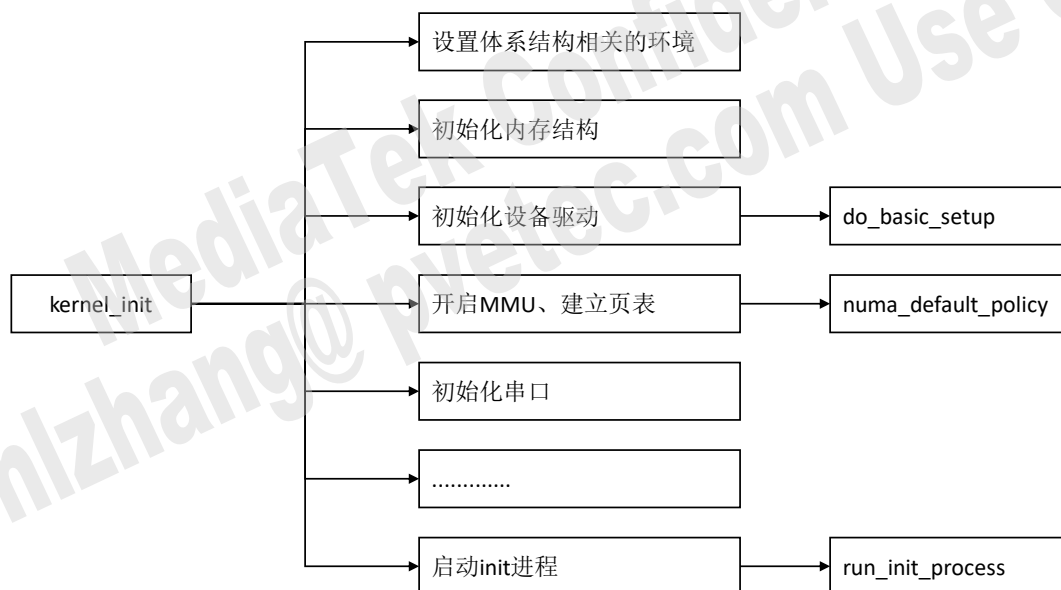


图1-12. kernel_init

1.7.4 initramfs

`initramfs` 是介于 `kernel` 和 `systemd` 之间的中间层，主要功能是加载一些基本的内核模块，为切换 `rootfs`、执行 `systemd` 做准备。`kernel` 初始化完成之后，首先运行 `initramfs` 下的 `init` 脚本。`init` 脚本会解析 `kernel` 启动参数，并导出为 `shell` 环境变量。然后调用 `/init.d` 目录下面的初始化脚本。在 MT8676 平台上，`/init.d` 目录下有 `10-kmod`、`11-partlabel`、`12-format`、`90-rootfs`、`99-finish` 等脚本。

`10-kmod` 会加载 `/etc/modules.list` 文件指定的内核模块。`/etc/modules.list` 是根据 `ko table` 文件生成的，如果

“Vendor/Ramdisk in Normal(vendor/ramdisk)” 列是 `ramdisk`，则 `/etc/modules.list` 文件会包含此内核模块。关于 `ko table` 格式，请参考 1.2.4.2 章节。`/etc/modules.list` 应该包含一些基础内核模块，例如 UFS 驱动等。

`11-partlabel` 负责解析分区名字，并在 `/dev/disk/by-partlabel` 下建立软链接。`12-format`、`90-rootfs` 等脚本依赖这些软链接。

`12-format` 负责格式化 `nvdta`、`protect1`、`protect2` 等分区。

`90-rootfs` 负责挂载 `rootfs` 分区到 `/rootfs` 目录。`rootfs` 分区由 `kernel` 启动参数 “`root=PARTLABEL=system`” 指定。

`99-finish` 负责在 `/rootfs` 目录挂载 `procfs`、`sysfs`、`devfs`，并将根目录切换到 `/rootfs`。最后启动 `systemd`。

1.7.5 systemd

`systemd` 是 Linux 系统上的一种系统和服务管理程序。相比于传统的 `SysVinit`，`systemd` 提供了并行启动、依赖管理、按需启动等现代化特性。

systemd 接管控制权后，首先执行一系列的预初始化任务，如设置默认的环境变量、初始化日志系统等。然后读取并解析 unit 文件，这些文件位于/etc/systemd/system、/lib/systemd/system 等目录。这些文件定了服务、挂载点、设备等资源的配置

根据 Unit 文件中的依赖关系，systemd 构建一个依赖树。然后，systemd 将尝试达到默认目标 /etc/systemd/system/default.target，这个目标由一组相互依赖的 unit 组成。systemd 会并行启动默认目标依赖的服务，并根据依赖树确保服务按正确的顺序启动。

表 1-16. 常见服务

服务	说明
systemd-modules-load	读取/etc/modules-load.d 下的配置文件并加载内核模块。/etc/modules-load.d/ 10-kernel-modules.conf 文件包含 ko table 中的 vendor ko。
systemd-udev	负责管理设置节点的创建和移除
android-tools-adbd	ADB 守护进程
weston	Weston 桌面服务
prop	属性服务进程。getprop 和 setprop 工具会和此服务交互。
mdpd_8676	MDP 后台服务进程
camerahalservice	Camera 服务进程
thermal-daemon	Thermal 守护进程

1.8 Yocto 常用调试工具

表 1-17. Yocto 常用调试工具

工具	说明
log	分析所有问题的第一手资料。关于 Yocto log 系统，请参考文档 <i>MTK_Yocto_Log_Tool_Guide</i> ，此文档可以在 Online 下载。
AEE	发生异常时，AEE 会收集相关调试信息到 db (datafile) 文件，是调试平台异常重启、kernel 异常、用户程序异常等问题的强大利器。请参考文档 <i>MTK_Yocto_AEE_Tool_Guide</i> ，此文档可以在 Online 下载。
kasan	用于调试 kernel 内存泄漏，请使用 auto8676p1_64_kasan project。除 kasan 外，还有 kmemleak、asan、valgrand 等调试内存泄漏的工具。
ftrace	抓取内核的 trace 日志，分析性能问题。
systemd-analyze	抓取 systemd 服务的启动时间，分析开机时间。

工具	说明
strace	抓取程序调用的系统调用。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。